Capstone Mini Project

Complete JavaScript Solution

This program demonstrates input validation, conditional logic, loops, functions, scope, and array methods using a product inventory.

JavaScript Code

```
"use strict";

// Capstone Mini Project:

// 1. Data & Input Initialization
const inventory = [
    { name: "Laptop", price: 800 },
    { name: "Mouse", price: 25 }
];

console.log("--- Initializing Inventory System ---");

// Get product details from the user
const productName = prompt("Enter new product name:");
if (productName === null || productName.trim() === "") {
    console.log("Invalid product name. Using default: Monitor");
}

const validName =
    productName && productName.trim() !== ""
        ? productName.trim()
        : "Monitor";

const priceInput = prompt("Enter product price:");
const enteredPrice = Number(priceInput);

const validPrice =
    priceInput !== null &&
    priceInput.trim() !== "" &&
    Number.isFinite(enteredPrice) &&
    enteredPrice > 0;

if (validPrice) {
    inventory.push({ name: validName, price: enteredPrice });
    console.log(
        `[Prompt executed: User adds "${validName}" at "$${enteredPrice}"]`
    );
} else {
    console.log("Invalid price. Adding default Monitor at $150.");
    inventory.push({ name: "Monitor", price: 150 });
}

// 2. Control Flow & Conditional Evaluation
function getCategory(price) {
    if (price < 50) {
        return "Budget";
    } else if (price < 200) {
        return "Standard";
    } else {
        return "Premium";
    }
}

function getCategoryBadge(price) {
    switch (getCategory(price)) {
        case "Budget":
            return "Budget";
        case "Standard":
            return "Standard";
        case "Premium":
            return "Premium";
        default:
            return "Unknown";
    }
}

// 3. Loops & Flow Control
console.log("\\n--- Processing Inventory Roster (forEach) ---");
```

```
inventory.forEach(function (item) {
    if (!item.name || !Number.isFinite(item.price)) {
        console.log("Skipping invalid inventory item.");
        return;
    }

    console.log(
        `* Item: ${item.name} ($${item.price}) -> Category: ${getCategoryBadge(item.price)}`
    );
});

// Search inventory using break and continue
const searchName = "Mouse";
let found = false;

for (const item of inventory) {
    if (!item.name || item.price <= 0) {
        continue;
    }

    if (item.name.toLowerCase() === searchName.toLowerCase()) {
        console.log(`Search result: ${item.name} found at $${item.price}`);
        found = true;
        break;
    }
}

if (!found) {
    console.log(`${searchName} not found.`);
}

// 4. Functions & Scope
function calculateTotal(items) {
    let total = 0;
    items.forEach(item => {
        total += item.price;
    });
    return total;
}

const calculateAverage = (items) => {
    return items.length
        ? calculateTotal(items) / items.length
        : 0;
};

// Local parameters do not modify the original inventory prices.
function applyAdjustment(items, taxRate = 0, discount = 0) {
    return items.map(item => {
        const adjustedPrice =
            item.price * (1 + taxRate / 100) * (1 - discount / 100);

        return {
            name: item.name,
            price: Number(adjustedPrice.toFixed(2))
        };
    });
}

// 5. Array Methods & Iteration
console.log("\\n--- Financial Analytics ---");

const totalValue = calculateTotal(inventory);
console.log(`Total Inventory Value: $${totalValue}`);

const affordableItems = inventory.filter(item => item.price < 200);
console.log(
    'Filtered Affordable Items (Under $200):',
    JSON.stringify(affordableItems.map(item => item.name))
);

const formattedReport = inventory.map(
    item => `${item.name.toUpperCase()}: $${item.price}`
);
console.log("Formatted Report List:", JSON.stringify(formattedReport));

console.log(`Average Product Price: $${calculateAverage(inventory).toFixed(2)}`);

const adjustedInventory = applyAdjustment(inventory, 10, 5);
console.log(
    "Adjusted Prices (10% tax, then 5% discount):",
    JSON.stringify(adjustedInventory)
```

```
);
```

Expected Console Output

```
--- Initializing Inventory System ---
[Prompt executed: User adds "Monitor" at "$150"]

--- Processing Inventory Roster (forEach) ---
* Item: Laptop ($800) -> Category: Premium
* Item: Mouse ($25) -> Category: Budget
* Item: Monitor ($150) -> Category: Standard
Search result: Mouse found at $25

--- Financial Analytics ---
Total Inventory Value: $975
Filtered Affordable Items (Under $200): ["Mouse","Monitor"]
Formatted Report List: ["LAPTOP: $800","MOUSE: $25","MONITOR: $150"]
Average Product Price: $325.00
Adjusted Prices (10% tax, then 5% discount): [{"name":"Laptop","price":836},{"name":"Mouse",
"price":26.13},{"name":"Monitor","price":156.75}]
```